DELIVERABLE 2 • REST & REAL-TIME INTERFACE REFERENCE

SalesPulse BI: API & WebSocket Documentation

Base REST URL: http://localhost:5000/api/v1 • Socket.io URL: http://localhost:5000 • Express 5 + BullMQ + Socket.io 4

1.1 API Conventions, Security & Rate Limiting

- **Base URLs & CORS:** All REST routes are mounted under `http://localhost:5000/api/v1` except health probes, which live at the server root (`http://localhost:5000/health`). Socket.io connects to `http://localhost:5000` . No authentication is enforced; CORS restricts browser origins to the comma-separated allowlist in `CLIENT_ORIGIN` (default `http://localhost:3000`).
- **Content Types & Body Limits:** JSON payloads are capped at `1 MB` . File uploads use `multipart/form-data` with field name `file` and are capped at `MAX_UPLOAD_MB` (default `50 MB`).
- **Standard Error Contract:** Every error returns `{ "message": string, "details"?: any }` . When `NODE_ENV !== 'production'` , `5xx` errors also include a `stack` trace. Status codes: `400` Invalid input/file, `404` Not found (unknown batch or route), `409` Conflict (deleting active job), `413` File too large, `429` Rate limited, `503` Dependency unavailable.
- **Rate Limiting:** Standard `RateLimit` headers are emitted. Global API limit: **1,500 requests / IP / 15 minutes** (`RATE_LIMIT_MAX`). Upload limit on `POST /uploads` : **30 uploads / IP / 15 minutes** (`UPLOAD_RATE_LIMIT_MAX`). Set `TRUST_PROXY=1` behind Nginx, Render, or Railway so rate limiters inspect real client IPs.

1.2 Health Probes & Dataset Batch Endpoints

Method	Endpoint Path	Status Codes	Request Contract & Response Behavior
GET	`/health`	`200`	**Liveness Probe (Root):** Exempt from rate limiting. Returns `{ "status": "ok", "uptime": <seconds> }` .
GET	`/health/ready`	`200` , `503`	**Readiness Probe (Root):** Returns `200 { status: "ready", database: "connected", redis }` or `503 { status: "unavailable", database: "disconnected", redis }` where `redis` is `connecting` , `connected` , or `reconnecting` . Only database connectivity determines readiness.
POST	`/api/v1/uploads`	`202` , `400` , `413` , `429` , `503`	**Upload & Queue Dataset:** Accepts `multipart/form-data` field `file` (`.csv` or `.xlsx`). Verifies extension, MIME type, and first magic bytes (`.xlsx` must be a valid ZIP header; `.csv` must be text with no binary bytes; empty files rejected). Enqueues BullMQ job within `ENQUEUE_TIMEOUT_MS` (5,000 ms). If Redis times out, marks batch `failed` , deletes temp file, and returns `503` .
GET	`/api/v1/uploads`	`200`	**List Batches:** Returns newest 200 uploads sorted by `uploadedAt` descending. Omits `errorLogs` for payload speed; adds aliases `batchId` (= `_id`) and `totalRows` (= `totalRowsRead`).
GET	`/api/v1/uploads/:batchId`	`200` , `400` , `404`	**Inspect Single Batch:** Returns full batch metadata including `errorLogs` formatted as `[{ rowNumber, field, message, reason }]` (`message` and `reason` carry identical text).
DELETE	`/api/v1/uploads/:batchId`	`200` , `400` , `404` , `409`	**Delete Batch & Records:** Deletes child `SalesRecord` rows, deletes the batch, and clears cache entries. Returns `200 { batchId, deletedRecords, message: "Upload deleted." }` . Returns `409` if status is

2.1 Analytics Query Parameters Reference

All four analytics endpoints under `/api/v1/analytics/*` require `batchId` and return `404` if the batch does not exist or `400` if any parameter fails validation.

Parameter	Endpoints	Required	Accepted Formats, Defaults & Validation Rules
`batchId`	All 4 endpoints	**Yes**	24-character hexadecimal MongoDB ObjectId string. Checked via regex prior to querying.
`startDate`	`overview`, `trends`, `breakdowns`	Optional	`YYYY-MM-DD` or ISO-8601 string. A bare date maps to the start of that UTC day (`00:00:00.000Z`). Ignored by `/forecast`.
`endDate`	`overview`, `trends`, `breakdowns`	Optional	`YYYY-MM-DD` or ISO-8601 string. A bare date includes the entire UTC day (`23:59:59.999Z`). Must be `≥ startDate` (returns `400` otherwise).
`category`	All 4 endpoints	Optional	Accepts comma-separated (`A,B`), repeated (`category=A&category=B`), or bracket (`category[]=A`) syntax up to 100 values. Category names containing commas must use repeated/bracket syntax. Object injections (`category[a]=b`) are rejected with `400`.
`region`	All 4 endpoints	Optional	Identical syntax, 100-item cap, and NoSQL operator injection guards as `category`.
`granularity`	`trends` only	Optional	One of `day`, `week` (ISO weeks labeled by their Monday `YYYY-MM-DD`), or `month` (`YYYY-MM`). Default: `month`.
`fillGaps`	`trends` only	Optional	Boolean (default `true`). Inserts zero-valued buckets for periods with no sales so quiet periods appear as dips. Pass `false` or `0` to disable. Automatically skipped if buckets exceed 2,000.

2.2 `GET /api/v1/analytics/overview` — KPIs & Growth Basis

Computes `totalRevenue` (sum of `totalRevenue`), `totalOrders` (distinct `orderId` count), `totalCustomers` (distinct `customerId` count), `averageOrderValue` (`totalRevenue / totalOrders`), and `revenueGrowth`. When both `startDate` and `endDate` are supplied, growth compares the selected window against the immediately preceding window of equal length (`basis: "previous_period"`); otherwise it compares the latest month in the filtered data against the prior month (`basis: "latest_month_vs_prior_month"`), returning `null` if prior revenue was zero. `filterOptions` (and legacy alias `options`) always describe the entire unfiltered batch so UI dropdowns never shrink.


```
{
  "kpis": {
    "totalRevenue": 25012345.67, "totalOrders": 950, "totalCustomers": 310,
    "averageOrderValue": 26328.78, "revenueGrowth": 4.82, "previousPeriodRevenue": 1980000.5
  },
  "growth": {
    "basis": "latest_month_vs_prior_month", "percent": 4.82,
    "currentRevenue": 2075000.1, "previousRevenue": 1980000.5,
    "currentPeriod": { "month": "2025-12" }, "previousPeriod": { "month": "2025-11" }
  },
  "filterOptions": { "categories": ["Accessories", "Electronics"], "regions": ["East",
"West"], "minDate": "2025-01-01", "maxDate": "2025-12-31" },
  "options": { "categories": ["Accessories", "Electronics"], "regions": ["East", "West"] }
}
```


2.3 `GET /api/v1/analytics/trends` — Time-Series Response

Returns a chronological array of bucketed revenue, distinct order counts, and average order value

2.4 `GET /analytics/breakdowns` & `GET /analytics/forecast`

`GET /api/v1/analytics/breakdowns` executes a single-pass `$facet` pipeline returning four dimensional arrays, emitting dual property aliases so both original backend keys and frontend adapter keys resolve cleanly. `GET /api/v1/analytics/forecast` accepts `batchId`, `category`, `region`, and `horizon` (integer `1–12`, default `3`; ignores date filters) to project future monthly revenue.

Endpoint	Response Property	Schema & Calculation Rules
`/breakdowns`	`topProducts`	Top 10 SKUs by revenue: `productId`, `productName`, `category`, `unitsSold`, `revenue` (aliased as `totalRevenue`), and `percentOfTotal` (aliased as `percentageOfSales`, share of filtered revenue).
`/breakdowns`	`topCustomers`	Top 10 buyers by spend: `customerId`, `customerName`, `totalOrders` (distinct orders), `totalSpend`, `averageOrderValue`, and `lastPurchaseDate` (ISO string).
`/breakdowns`	`categoryDistribution`	Every category in filtered data: `category`, `name` (alias), `revenue`, `percentOfTotal`, and `percentage` (alias).
`/breakdowns`	`regionDistribution`	Every region in filtered data: `region`, `name` (alias), `revenue`, `percentOfTotal`, and `percentage` (alias).
`/forecast`	`historical`	`[{ month: "YYYY-MM", revenue: number }]` sorted chronologically with empty intermediate months zero-filled.
`/forecast`	`forecast`	`[{ month: "YYYY-MM", revenue: number, lower: number | null, upper: number | null }]` where `lower` and `upper` form a 95% prediction range (`null` when history has < 3 months).
`/forecast`	`method`, `methodUsed`	`"linear_wma_blend"` (≥ 3 months: 70% OLS + 30% 3-month WMA), `"sma"` (1–2 months), or `"insufficient_data"` (0 months).
`/forecast`	`trend`, `trendDirection`	`"upward"`, `"downward"`, or `"flat"` (`flat` when regression slope is within ±1% of average monthly revenue).
`/forecast`	`slope`, `intercept`	Ordinary Least Squares linear regression coefficients rounded to 2 decimal places.
`/forecast`	`rSquared`, `confidence`	`rSquared` in `[0, 1]`. Server sets `confidence` to `"High"` ($R^2 \geq 0.7$), `"Medium"` ($R^2 \geq 0.4$), or `"Low"` (< 0.4).
`/forecast`	`dataPoints`, `horizon`, `intervalConfidence`, `note`	Count of historical months used, requested forecast horizon (`1–12`), `0.95` confidence interval level, and an explanatory note when history is thin.

2.5 Forecast Payload Example

```
{
  "historical": [ { "month": "2025-10", "revenue": 1840200 }, { "month": "2025-11",
"revenue": 1980000.5 }, { "month": "2025-12", "revenue": 2075000.1 } ],
  "forecast": [ { "month": "2026-01", "revenue": 2168400.25, "lower": 2010500.10, "upper":
2326300.40 } ],
  "method": "linear_wma_blend", "methodUsed": "linear_wma_blend",
  "trend": "upward", "trendDirection": "upward",
  "slope": 117400.05, "intercept": 1843266.82, "rSquared": 0.98, "confidence": "High",
  "dataPoints": 3, "horizon": 1, "intervalConfidence": 0.95, "note": null
}
```

2.6 In-Memory Analytics Cache Behavior

Deterministic TTL Caching & Request Coalescing (utils/cache.js)

3.1 Real-Time WebSocket Events (Socket.io)

Clients connect to `http://localhost:5000` using `socket.io-client`. The Express API listens to BullMQ's Redis `QueueEvents` stream and relays progress, completion, and failure events to connected browsers, working identically whether the worker runs inside the API process (`RUN_WORKER_IN_API=true`) or as a separate process (`npm run worker`).

Direction	Event Name	Payload Schema & Delivery Semantics
`Client → Server`	`join:batch`	Payload: `batchId` (`string`). Joins the socket to the batch room and **immediately replays** the batch's current database state (`upload:progress`, `upload:completed`, or `upload:failed`) so fast jobs that finish before subscription are never missed.
`Client → Server`	`leave:batch`	Payload: `batchId` (`string`). Leaves the batch room when the upload component unmounts or switches datasets.
`Server → Client`	`upload:progress`	Payload: `{ batchId, progress, processedRows, totalRows? }`. `progress` is an integer `0–100`. For streamed CSVs, `totalRows` is unknown during streaming so progress is calculated from bytes read and never reports `100` until completion.
`Server → Client`	`upload:completed`	Payload: `{ batchId, fileName, inserted, skipped, totalRows, errorLogs, errorLogsTruncated, dateOrder, notes }`. Emitted once all chunks are committed to MongoDB.
`Server → Client`	`upload:failed`	Payload: `{ batchId, reason, error }` (both fields carry the same user-safe failure message). Emitted after partial inserts are rolled back.

3.2 Step-by-Step Postman & Local Verification Guide

1. **Start Infrastructure & Backend:** Ensure MongoDB (port `27017`) and Redis (port `6379`) are running. In `server/`, run `cp .env.example .env`, `npm install`, and `npm run dev`. Verify healthy boot via `GET http://localhost:5000/health/ready`.
2. **Upload a Dataset (`POST /api/v1/uploads`):** In Postman, set method to `POST` and URL to `http://localhost:5000/api/v1/uploads`. Do not set a manual `Content-Type` header. Open **Body → form-data**, enter key `file`, change the key type dropdown from *Text* to *File*, and select `sample-data/MOCK_DATA(1).csv`. Send the request and copy the returned `batchId` from the `202 Accepted` JSON response.
3. **Inspect Ingestion Status & Quarantine Logs (`GET /api/v1/uploads/:batchId`):** Query `http://localhost:5000/api/v1/uploads/<batchId>`. Confirm `status: "completed"`, `validRowsInserted: 1000`, `dateOrder: "mdy"`, and inspect `errorLogs` for any skipped rows.
4. **Query KPI Overview (`GET /api/v1/analytics/overview`):** Pass query params `batchId=<batchId>` and optional `startDate=2025-01-01&endDate=2025-12-31`. Verify the 5 KPIs, growth object, and `filterOptions`.
5. **Query Revenue Trends (`GET /api/v1/analytics/trends`):** Pass `batchId=<batchId>&granularity=month&fillGaps=true` to retrieve the zero-filled chronological time series.
6. **Query Dimensional Breakdowns (`GET /api/v1/analytics/breakdowns`):** Pass `batchId=<batchId>` (and optional `category=Electronics,Furniture&region=West`) to retrieve `topProducts`, `topCustomers`, `categoryDistribution`, and `regionDistribution`.
7. **Query Sales Forecast (`GET /api/v1/analytics/forecast`):** Pass `batchId=<batchId>&horizon=3` to inspect historical monthly totals, 3-month blended projections with 95% `lower` / `upper` intervals, `slope`, and `rSquared`.

> **Broadcast vs. Room-Scoped WebSockets (SOCKET_ROOMS)**
>
> By default (`SOCKET_ROOMS=false`), server events are broadcast to all connected browsers and the client filters events by `batchId`. Setting `SOCKET_ROOMS=true` restricts event delivery strictly to sockets that emitted `join:batch` for that `batchId`.